

CLAUDE CODE STRATEGY

# 今 Claude Code を選ぶべき理由

拡張性 × CLI × 最新エコシステム

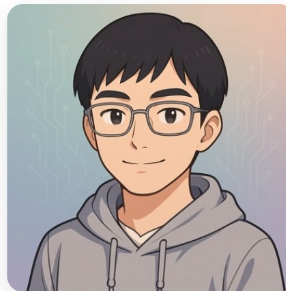
開発にどう組み込む？AIエージェント選定ガイド  
～Claude Code / Cursor / GitHub Copilot 実務比較～

スナガク

# 自己紹介

## スナガク

- ソフトウェアエンジニア
- 趣味: 個人開発 サウナ
- 最近は Claude Code にハマリ中



## 個人開発

- AIに気軽に恋愛相談できるアプリ **Lovady**
- 「今やるべきこと」を明確にするアプリ **FocusOne**



<https://lovady.app/>



<https://focusone.app/>

# 目次

**Cursor / Copilot と比べた Claude Code の特徴**

**スナガクがおすすめする使い方**

# Cursor / Copilot と比べた Claude Code の特徴

# CLI で提供されている

## Cursor ・ GitHub Copilot

IDE に統合されている

CLI はあるが、まだ発展途上

## Claude Code

Terminal だけで実装が完結

複数起動しても認知負荷が低く、  
並列作業しやすい

...でも CLI だけで大丈夫？ 変更差分が見えないと不安じゃない？

# Plan モードで精度の高い実装計画を作成可能

## 課題

AI に大規模タスクを任せると、思い通りの実装をしてくれない

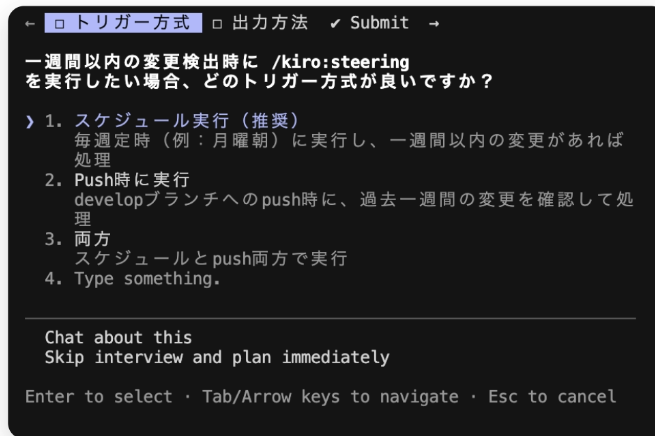
## 解決策

「事前に実装計画書を作成」できると、AI の実装内容を予測可能に

## ポイント

「分からない情報・不明確な要件は質問して」と伝えると、Claude から質問してくれる

→ 曖昧だった仕様が明確になる



# 豊富な機能 × 蓄積されたノウハウ

## 豊富な機能

サブエージェント・Skills・カスタムコマンド etc...

ワークフローを柔軟に構築できる

## 蓄積されたノウハウ

Zenn での記事数が多い → 困ったときに情報を探しやすい



### GitHub Copilot

このトピックを指定するには `githubcopilot` と入力

Articles 503 Books 5 Scraps 54

GitHub Copilot



### Cursor

このトピックを指定するには `cursor` と入力

Articles 1175 Books 8 Scraps 83

Cursor



### Claude Code

このトピックを指定するには `claudecode` と入力

Articles 1950 Books 6 Scraps 131

Claude Code

# 最新技術、実は Anthropic 発が多い！？

## Skills や MCP は Anthropic 発

Claude Code を使えば 最新技術をキャッチアップできる

Announcements

### Introducing the Model Context Protocol

2024年11月26日

Today, we're open-sourcing the [Model Context Protocol \(MCP\)](#), a new standard for connecting AI assistants to the systems where data lives, including content repositories, business tools, and development environments. Its aim is to help frontier models produce better, more relevant responses.

As AI assistants gain mainstream adoption, the industry has invested heavily in model capabilities, achieving rapid advances in reasoning and quality. Yet even the most sophisticated models are constrained by their isolation from data—trapped behind information silos and legacy systems. Every new data source requires its own custom implementation, making truly connected systems difficult to scale.

MCP addresses this challenge. It provides a universal, open standard for connecting AI systems with data sources, replacing fragmented integrations with a single protocol. The result is a simpler, more reliable way to give AI systems access to the data they need.

#### Model Context Protocol

The Model Context Protocol is an open standard that enables developers to build secure, two-way connections between their data sources and AI-powered tools. The architecture is straightforward: developers can either expose their data through MCP servers or build AI applications (MCP clients) that connect to these servers.

Today, we're introducing three major components of the Model Context Protocol:

<https://www.anthropic.com/news/model-context-protocol>



### Equipping agents for the real world with Agent Skills

Published Oct 16, 2025

Claude is powerful, but real work requires procedural knowledge and organizational context. Introducing **Agent Skills**, a new way to build specialized agents using files and folders.

*Update: We've published [Agent Skills](#) as an open standard for cross-platform portability. (December 18, 2025)*

As model capabilities improve, we can now build general-purpose agents that interact with full-fledged computing environments. **Claude Code**, for example, can accomplish complex tasks across domains using local code execution and filesystems. But as these agents become more powerful, we need more composable, scalable, and portable ways to equip them with domain-specific expertise.

This led us to create **Agent Skills**: organized folders of instructions, scripts, and resources that agents can discover and load dynamically to perform better at specific tasks. Skills extend Claude's capabilities by packaging your expertise.

<https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>

スナガクがおすすめする使い方

# Git Worktree × Claude Code で並列作業

## Git Worktree とは

1つのリポジトリで複数ブランチを別ディレクトリで同時に作業できる機能

## 事前に実装計画を立てれば、実装は AI に任せられる

その間は、自分は別タスクを進められる

## メイン作業をしつつ、隙間時間で細かいタスクも消化可能

後回しにしがちなタスクも一緒に進められる

# 定型作業・AI がミスしやすいタスクは Skill 化

## Skills とは

プロンプトをファイル化して再利用できる機能  
Codex など他ツールでも使える共通規格

## Skills の特徴：段階的な開示

初めはスキル名や概要だけ読ませ、必要に応じて深く参照  
→ CLAUDE.md に全情報を詰め込む必要がない

## Skills に記載する内容：判断軸の明確化

AI が迷わず動ける行動指針を設定

## 活用例

ビルド手順、エラー対応、テスト作成  
Figma → コード化 Skills を導入・改善中  
→ チーム全員でメリットを享受

## 注意点

現時点では明示的に呼び出す必要あり  
例：「figma-to-code のスキル使って」・「/figma-to-code」など

# コードの理解・キャッチアップに活用

## コードの中身について質問

実装の意図や処理の流れを Claude に聞ける

## 新規参画時のキャッチアップ

コードの全体像やアーキテクチャを把握しやすい

## ライブラリの中身まで読み込める

技術とライブラリの知識を紐付けて理解できる

## 新しい技術でも動くものが作れる

そこから質問・確認して素早くキャッチアップ

## 活用例

node\_modules のライブラリの中の処理を質問  
ライブラリの中身まで確認して答えてくれる  
→ 技術の理解が深まる

## PoC 作成・キャッチアップの流れ

- ① AI に動くものを作らせる
- ② コードと技術の対応を把握
- ③ 理解した上でリファクタ・作り直し

# バッチ処理で Claude Code を実行する

## PR 自動レビュー

Claude がコードをチェック

## ドキュメント自動更新

変更差分から自動で反映

## リファクタリング検出

候補を自動で Issue 化

## PR コメントで Claude 実行

- ・ レビュー指摘をそのまま修正依頼
- ・ CI エラーもローカル不要で修正

# まとめ

Claude Code は CLI で Terminal だけで完結

Plan モードで精度の高い実装計画を作成可能

Git Worktree と組み合わせて並列作業

Skills でチーム全体の生産性向上

バッチ処理で自動化の幅が広がる

ご清聴ありがとうございました！



@suna\_gaku

よかったら X フォローしてください！